บทที่ 10: ความรู้เบื้องต้นเกี่ยวกับแพ็กเกจการจัดการ State (Provider)

10.1 บทนำ: ความท้าทายของ setState ในแอปพลิเคชันขนาดใหญ่

ในบทที่ 7 เราได้เรียนรู้วิธีจัดการกับ state ที่เปลี่ยนแปลงได้ภายในวิดเจ็ตเดียวโดยใช้ StatefulWidget และ setState(). นี่เป็นกลไกพื้นฐานและทรงพลังสำหรับการจัดการ *ephemeral state* (state เฉพาะที่) เช่น ค่าปัจจุบันของตัวนับ หรือสถานะการเลือกของ checkbox.

อย่างไรก็ตาม ลองจินตนาการถึงแอปพลิเคชันที่ซับซ้อนขึ้น:

- **State ที่ต้องแชร์ข้ามหน้าจอ:** ข้อมูลการล็อกอินของผู้ใช้, ธีมของแอป, หรือเนื้อหาในรถเข็นสินค้า จำเป็นต้องเข้าถึงได้จากหลายๆ ส่วนของแอปที่อาจไม่ได้เชื่อมต่อกันโดยตรงใน widget tree.
- **Widget Tree ที่ลึก:** การส่ง state ลงไปหลายๆ ชั้นโดยใช้ constructor arguments และการส่ง callbacks กลับขึ้นมาหลายๆ ชั้นเพื่อแก้ไข state ("lifting state up") กลายเป็นเรื่องยุ่งยาก, เกิดข้อผิดพลาดได้ง่าย, และทำให้โค้ดอ่านยาก.
- **การแยก Concerns:** การผสมตรรกะทางธุรกิจ (business logic) หรือการจัดการ state เข้ากับโค้ด UI โดยตรง (ภายใน setState) ทำให้การทดสอบ, การบำรุงรักษา, และการทำความเข้าใจโค้ดยากขึ้น.

เมื่อแอปของคุณเติบโตขึ้น การพึ่งพา setState เพียงอย่างเดียวสำหรับ state ทั้งหมด (โดยเฉพาะ *app state* หรือ state ที่แชร์) จะนำไปสู่ปัญหาเหล่านี้. เราต้องการวิธีที่มีโครงสร้าง, ปรับขนาดได้, และเป็นระเบียบมากขึ้นในการจัดการและเข้าถึง state จากส่วนต่างๆ ของแอปพลิเคชัน.

นี่คือจุดที่ **แพ็กเกจการจัดการ State (State Management Packages)** เข้ามามีบทบาท. แพ็กเกจเหล่านี้มอบรูปแบบและเครื่องมือที่กำหนดไว้ล่วงหน้าเพื่อช่วยจัดการ state ได้อย่างมีประสิทธิภาพมากขึ้น. มีแพ็กเกจยอดนิยมหลายตัวในระบบนิเวศของ Flutter (เช่น Riverpod, BLoC/Cubit, GetX, MobX) แต่ละตัวมีแนวทางและข้อดีข้อเสียต่างกันไป.

ในบทนี้ เราจะแนะนำหนึ่งในแพ็กเกจที่เข้าถึงง่ายและได้รับความนิยมมากที่สุด: **Provider**. [1] Provider สร้างขึ้นบนพื้นฐานของ InheritedWidget ของ Flutter (กลไกพื้นฐานสำหรับการส่งข้อมูลลงไปใน tree) แต่มอบ API ที่ใช้งานง่ายกว่าสำหรับการจัดการ state ประเภทต่างๆ.

เราจะมาดูกันว่า Provider ทำงานอย่างไร, แนวคิดหลักของมันคืออะไร (ChangeNotifier, ChangeNotifierProvider, Consumer, context.watch, context.read), และวิธีใช้มันเพื่อ refactor แอปพลิเคชันตัวนับที่เราสร้างขึ้นก่อนหน้านี้.

10.2 ทำไมต้องใช้ State Management Package?

ก่อนที่จะเจาะลึก Provider เรามาสรุปประโยชน์หลักของการใช้ state management package กันก่อน:

1. **การแยก Concerns (Separation of Concerns):** แยกตรรกะการจัดการ state ออก

จากโค้ด UI ทำให้โค้ดสะอาดขึ้น, ทดสอบง่ายขึ้น, และบำรุงรักษาง่ายขึ้น. UI มีหน้าที่แสดง state และส่งต่อการกระทำของผู้ใช้, ส่วนตรรกะ state จัดการข้อมูลและแจ้ง UI เมื่อมีการเปลี่ยนแปลง.

2. **การเข้าถึง State ที่ง่ายขึ้น:** ทำให้วิดเจ็ตต่างๆ ใน tree สามารถเข้าถึง state ที่ต้องการได้ง่ายขึ้น โดยไม่ต้องส่งข้อมูลผ่าน constructor หรือ callbacks หลายชั้น.
3. **ลด Boilerplate Code:** ลดจำนวนโค้ดที่ต้องเขียนซ้ำๆ สำหรับการส่ง state และ callbacks ผ่าน widget tree.
4. **ปรับปรุงประสิทธิภาพ:** แพ็กเกจจำนวนมากมีกลไกในการ rebuild เฉพาะวิดเจ็ตที่จำเป็นต้องอัปเดตเมื่อ state เปลี่ยนแปลง แทนที่จะ rebuild ทั้ง subtree ขนาดใหญ่.
5. **ความสามารถในการปรับขนาด (Scalability):** ทำให้การจัดการ state ในแอปพลิเคชันขนาดใหญ่และซับซ้อนง่ายขึ้น.

10.3 ขอแนะนำ Provider

Provider เป็น "wrapper รอบ InheritedWidget เพื่อให้ใช้งานง่ายขึ้นและนำกลับมาใช้ใหม่ได้มากขึ้น" [1]. มันไม่ใช่โซลูชันการจัดการ state แบบครบวงจรในตัวเอง แต่เป็นเฟรมเวิร์กสำหรับการจัดการ dependency injection และ state management โดยใช้ประโยชน์จากแนวคิดที่มีอยู่ของ Flutter.

แนวคิดหลักใน Provider ที่เราจะเน้นในบทนี้คือ:

1. **ChangeNotifier**: คลาสพื้นฐานใน Flutter (ไม่ใช่ส่วนหนึ่งของ Provider โดยตรง แต่ Provider ใช้ประโยชน์จากมันอย่างมาก) ที่ให้กลไกการแจ้งเตือนการเปลี่ยนแปลง (change notification) แก่ listeners. คุณจะสร้างคลาส state model ของคุณโดย extends ChangeNotifier. [2]
2. **ChangeNotifierProvider**: วิดเจ็ตจากแพ็กเกจ Provider ที่ *ให้ (provides)* อินสแตนซ์ของ ChangeNotifier ของคุณแก่ children ของมันใน widget tree. วิดเจ็ตที่อยู่ใต้ ChangeNotifierProvider สามารถเข้าถึงอินสแตนซ์นั้นได้. [3]
3. **การเข้าถึง State (Consuming)**: มีหลายวิธีที่วิดเจ็ตจะเข้าถึง (consume) state ที่ถูก provide:
    - **context.watch<T>()**: วิธีที่แนะนำสำหรับการเข้าถึงค่า state *และ* ทำให้วิดเจ็ต rebuild เมื่อ state นั้น (ที่ได้จาก ChangeNotifier) มีการเปลี่ยนแปลง (เรียก notifyListeners()). ควรใช้ภายในเมธอด build. [4]
    - **context.read<T>()**: เข้าถึงค่า state *โดยไม่* ทำให้วิดเจ็ต rebuild เมื่อ state เปลี่ยนแปลง. เหมาะสำหรับการอ่านค่า state เพียงครั้งเดียว หรือการเรียกเมธอดบน provider ภายใน event handlers (เช่น onPressed). ไม่ควรใช้ภายในเมธอด build หากคุณต้องการให้ UI อัปเดตตาม state นั้น. [4]
    - **Consumer<T>**: วิดเจ็ตที่รับ builder function ซึ่งจะถูกเรียกใหม่เมื่อ ChangeNotifier ที่ระบุ (T) มีการเปลี่ยนแปลง. ช่วยให้คุณสามารถ rebuild เฉพาะส่วนเล็กๆ ของ widget tree แทนที่จะ rebuild ทั้งเมธอด build ของวิดเจ็ตแม่. [5]
    - **context.select<T, R>(R Function(T value) selector)**: คล้ายกับ watch แต่ให้คุณเลือกเฉพาะส่วนของ state ที่วิดเจ็ตสนใจ. วิดเจ็ตจะ rebuild ก็ต่อเมื่อค่าที่ *เลือก* ออกมานั้น

เปลี่ยนแปลงเท่านั้น ช่วยเพิ่มประสิทธิภาพ. [6]

เราจะใช้ ChangeNotifier, ChangeNotifierProvider, context.watch, และ context.read ในแล็บของเรา.

10.4 การทำงานของ Provider โดยสังเขป

1. **สร้าง State Model:** คุณสร้างคลาส (เช่น CounterModel) ที่ extends ChangeNotifier. คลาสนี้เก็บ state (เช่น int count) และเมธอดสำหรับแก้ไข state (เช่น increment()). ภายในเมธอดที่แก้ไข state, คุณต้องเรียก notifyListeners() เพื่อแจ้งให้ผู้ฟังทราบว่ามีบางอย่างเปลี่ยนแปลง.
2. **Provide State:** ในตำแหน่งที่เหมาะสมใน widget tree (โดยทั่วไปคือเหนือวิดเจ็ตทั้งหมดที่ต้องการเข้าถึง state นั้น), คุณห่อส่วนของ tree ด้วย ChangeNotifierProvider<CounterModel>. คุณสร้างอินสแตนซ์ของ CounterModel และส่งให้ ChangeNotifierProvider.
3. **Consume State:**
   - ในวิดเจ็ตที่ต้องการ *แสดง* state ปัจจุบัน (เช่น Text ที่แสดงค่า count), คุณใช้ context.watch<CounterModel>() ภายในเมธอด build เพื่อรับอินสแตนซ์ CounterModel และเข้าถึงค่า state (เช่น counterModel.count). วิดเจ็ตนี้จะ rebuild โดยอัตโนมัติเมื่อ CounterModel เรียก notifyListeners().
   - ในวิดเจ็ตที่ต้องการ *กระตุ้น* การเปลี่ยนแปลง state (เช่น ElevatedButton), คุณใช้ context.read<CounterModel>() ภายใน callback onPressed เพื่อรับอินสแตนซ์ CounterModel และเรียกเมธอดของมัน (เช่น counterModel.increment()).

สิ่งนี้จะแยก state และตรรกะการเปลี่ยนแปลงออกจาก UI ทำให้โค้ดเป็นระเบียบมากขึ้น.

10.5 การติดตั้ง Provider

ในการใช้ Provider คุณต้องเพิ่มมันเป็น dependency ในไฟล์ pubspec.yaml ของคุณ:

```
dependencies:
  flutter:
    sdk: flutter
  provider: ^6.1.2 # ตรวจสอบเวอร์ชันล่าสุดบน pub.dev เสมอ

  # dependencies อื่นๆ ของคุณ...
```

หลังจากเพิ่มบรรทัดนี้แล้ว ให้บันทึกไฟล์ และรัน flutter pub get ใน terminal (หรือรอให้ IDE ของคุณทำโดยอัตโนมัติ).

10.6 สรุปบทเรียนและสิ่งที่จะเรียนรู้ต่อไป

ในส่วนบรรยายนี้ เราได้ทำความเข้าใจถึงข้อจำกัดของ setState สำหรับ state ที่ซับซ้อนหรือแชร์กัน และได้แนะนำแนวคิดของการจัดการ state โดยใช้แพ็กเกจภายนอก โดยเน้นที่ Provider:

- **ปัญหาของ setState:** ความยุ่งยากในการ "lifting state up", การผสม UI และตรรกะ, และความยากลำบากในการแชร์ state ข้ามส่วนต่างๆ ของแอป.
- **ประโยชน์ของ State Management Packages:** การแยก concerns, การเข้าถึง state ที่ง่ายขึ้น, ลด boilerplate, และปรับปรุงประสิทธิภาพ/ความสามารถในการปรับขนาด.
- **Provider คืออะไร:** Wrapper รอบ InheritedWidget ที่ใช้งานง่าย.
- **แนวคิดหลักของ Provider (สำหรับ ChangeNotifier):**
    - ChangeNotifier: คลาสสำหรับ state model ที่สามารถแจ้งเตือน listeners.
    - ChangeNotifierProvider: วิดเจ็ตสำหรับ "provide" อินสแตนซ์ ChangeNotifier ให้กับ tree.
    - context.watch<T>(): เข้าถึง state และ rebuild เมื่อมีการเปลี่ยนแปลง.
    - context.read<T>(): เข้าถึง state/เมธอดโดยไม่ rebuild.
    - Consumer<T> / context.select<T, R>: สำหรับการ rebuild ที่เฉพาะเจาะจงมากขึ้น (เพิ่มประสิทธิภาพ).
- **ขั้นตอนการติดตั้ง:** เพิ่ม dependency ใน pubspec.yaml.

ตอนนี้เรามีความเข้าใจในเชิงทฤษฎีแล้ว ส่วนปฏิบัติการถัดไปจะแสดงวิธีนำแนวคิดเหล่านี้ไปใช้จริงโดยการ refactor แอปพลิเคชันตัวนับของเรา.

บทต่อไป:

เมื่อเรามีวิธีจัดการ state ที่ดีขึ้นแล้ว เราพร้อมที่จะโต้ตอบกับโลกภายนอกมากขึ้น. **บทที่ 11: Asynchronous Programming & Networking** จะสอนวิธีทำงานกับการดำเนินการที่ไม่พร้อมกัน (asynchronous operations) เช่น การดึงข้อมูลจากอินเทอร์เน็ต (networking) โดยใช้ Future, async, await และแพ็กเกจ http.

เอกสารอ้างอิง (References)

[หมายเหตุ: วันที่เข้าถึงถูกตั้งค่าเป็น 30 เมษายน 2025 แหล่งอ้างอิงส่วนใหญ่เป็นภาษาอังกฤษ]

1. Flutter Team. (n.d.). *Provider package*. Pub.dev. Retrieved April 30, 2025, from https://pub.dev/packages/provider
2. Flutter Team. (n.d.). *ChangeNotifier class*. Flutter API Docs. Retrieved April 30, 2025, from https://api.flutter.dev/flutter/foundation/ChangeNotifier-class.html
3. Flutter Team. (n.d.). *ChangeNotifierProvider class*. Provider package API Docs. Retrieved April 30, 2025, from https://pub.dev/documentation/provider/latest/provider/ChangeNotifierProvider-class.html
4. Flutter Team. (n.d.). *Reading values*. Provider package Docs. Retrieved April 30, 2025, from https://pub.dev/documentation/provider/latest/provider/Reading%20values-topic.html (Covers watch, read, select)
5. Flutter Team. (n.d.). *Consumer class*. Provider package API Docs. Retrieved April 30, 2025, from https://pub.dev/documentation/provider/latest/provider/Consumer-class.html
6. Flutter Team. (n.d.). *BuildContext extensions - select method*. Provider package API Docs. Retrieved April 30, 2025, from https://pub.dev/documentation/provider/latest/provider/BuildContext/select.html

ปฏิบัติการบทที่ 10: Refactor ตัวนับด้วย Provider

วัตถุประสงค์: เพื่อฝึกฝนการ refactor แอปพลิเคชันที่ใช้ setState แบบง่ายๆ ให้ใช้แพ็กเกจ provider สำหรับการจัดการ state โดยใช้ ChangeNotifier, ChangeNotifierProvider, context.watch, และ context.read.

เวลาโดยประมาณ: 2 ชั่วโมง

สิ่งที่ต้องมี:

- ความเข้าใจใน StatelessWidget, StatefulWidget, และ setState (จากบทที่ 7).
- โปรเจกต์ Flutter ที่ตั้งค่าไว้ (สามารถสร้างใหม่หรือใช้จากแล็บก่อนหน้า).
- ติดตั้ง VS Code (หรือ Android Studio).
- Emulator หรืออุปกรณ์จริงกำลังทำงาน.

ส่วนที่ 1: ตั้งค่าโปรเจกต์และ Dependencies

1. **คำแนะนำ:** สร้างโปรเจกต์ Flutter ใหม่ (หรือเปิดโปรเจกต์ที่มีอยู่). หากใช้โปรเจกต์เก่า ให้แน่ใจว่าคุณมีโครงสร้างพื้นฐาน main.dart ที่มี MyApp (Stateless) และ MyHomePage (อาจเป็น Stateless หรือ Stateful ในตอนแรก). เราจะสร้างแอปตัวนับขึ้นมาใหม่หรือ refactor อันเก่า.
    - **ตัวอย่าง main.dart เริ่มต้น (หากสร้างใหม่):**

```
// lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Provider Counter Lab',
      theme: ThemeData(
        primarySwatch: Colors.indigo, // เปลี่ยนสี Theme ตามชอบ
      ),
      home: const MyHomePage(title: 'Provider Counter'),
    );
```

```dart
  }
}

// เราจะสร้าง MyHomePage ให้เป็น Stateful ก่อน แล้วค่อย refactor
class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key, required this.title});
  final String title;

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  int _counter = 0; // State เดิมที่ใช้ setState

  void _incrementCounter() {
    setState(() {
      _counter++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(widget.title),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            const Text(
              'You have pushed the button this many times:',
            ),
            Text(
              '$_counter', // แสดง state เดิม
              style: Theme.of(context).textTheme.headlineMedium,
            ),
```

```
          ],
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: _incrementCounter, // เรียกใช้ setState เดิม
        tooltip: 'Increment',
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

- **ผลลัพธ์ที่คาดหวัง (UI เริ่มต้น):** แอปตัวนับมาตรฐานที่ทำงานโดยใช้ setState.

2. **คำแนะนำ:** เปิดไฟล์ pubspec.yaml.
3. **คำแนะนำ:** ใต้ส่วน dependencies:, เพิ่ม provider: ตามด้วยเวอร์ชันล่าสุด (ตรวจสอบได้จาก pub.dev).

```
dependencies:
  flutter:
    sdk: flutter
  provider: ^6.1.2 # หรือเวอร์ชันล่าสุด
  # cupertino_icons อาจมีอยู่แล้ว...
```

4. **คำแนะนำ:** บันทึกไฟล์ pubspec.yaml. VS Code ควรจะรัน flutter pub get โดยอัตโนมัติ หากไม่ ให้รันด้วยตนเองใน terminal.
   - **ผลลัพธ์ที่คาดหวัง (Console):** ข้อความแสดงว่า provider ถูกดาวน์โหลดและติดตั้งสำเร็จ (เช่น Resolving dependencies... Got dependencies!).

ส่วนที่ 2: สร้าง ChangeNotifier Model

1. **คำแนะนำ:** ในโฟลเดอร์ lib, สร้างโฟลเดอร์ใหม่ชื่อ providers (หรือ models, state ตามชอบ).
2. **คำแนะนำ:** ภายในโฟลเดอร์ providers, สร้างไฟล์ใหม่ชื่อ counter_provider.dart.
3. **คำแนะนำ:** ใน counter_provider.dart, import package:flutter/foundation.dart (ซึ่งมี ChangeNotifier).
4. **คำแนะนำ:** สร้างคลาสชื่อ CounterProvider ที่ extends ChangeNotifier.

```dart
// lib/providers/counter_provider.dart
import 'package:flutter/foundation.dart';

class CounterProvider extends ChangeNotifier {
  // State และ Logic จะมาที่นี่
}
```

5. **คำแนะนำ:** ย้ายตัวแปร state _counter จาก _MyHomePageState มาไว้ใน CounterProvider. ทำให้เป็น private (_count) และสร้าง getter สาธารณะ (count) เพื่อให้เข้าถึงได้จากภายนอก (เป็นแนวปฏิบัติที่ดี). กำหนดค่าเริ่มต้นเป็น 0.

```dart
class CounterProvider extends ChangeNotifier {
  int _count = 0; // State เป็น private

  int get count => _count; // Getter สาธารณะสำหรับอ่านค่า
}
```

6. **คำแนะนำ:** ย้ายตรรกะการเพิ่มค่า (_incrementCounter) จาก _MyHomePageState มาไว้ใน CounterProvider. เปลี่ยนชื่อเมธอดเป็น increment (หรือชื่ออื่นที่เหมาะสม).
7. **คำแนะนำ: สำคัญ:** ภายในเมธอด increment, หลังจากเพิ่มค่า _count แล้ว ให้เรียก notifyListeners();. นี่คือสิ่งที่บอก Provider และวิดเจ็ตที่กำลังฟังอยู่ว่า state มีการเปลี่ยนแปลง.

```dart
class CounterProvider extends ChangeNotifier {
  int _count = 0;
  int get count => _count;

  void increment() {
    _count++;
    notifyListeners(); // แจ้งเตือน listeners ว่า state เปลี่ยนแปลง!
  }
}
```

- **ผลลัพธ์ที่คาดหวัง (Code):** คุณมีคลาส CounterProvider ที่ห่อหุ้ม state ของตัวนับและตรรกะการเพิ่มค่า พร้อมเรียก notifyListeners อย่างถูกต้อง.

ส่วนที่ 3: Provide the State

1. **คำแนะนำ:** กลับไปที่ main.dart.
2. **คำแนะนำ:** Import CounterProvider และแพ็กเกจ provider.

```
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart'; // Import provider
import 'providers/counter_provider.dart'; // Import model ของเรา
// ... (คลาส MyApp, MyHomePage, _MyHomePageState ยังคงเหมือนเดิมในตอนนี้)
...
```

3. **คำแนะนำ:** ค้นหาฟังก์ชัน main(). แก้ไขการเรียก runApp() โดยห่อ MyApp() ด้วย ChangeNotifierProvider.
4. **คำแนะนำ:** กำหนด create callback ให้กับ ChangeNotifierProvider. callback นี้รับ context และควรส่งคืนอินสแตนซ์ *ใหม่* ของ CounterProvider.

```
void main() {
  runApp(
    ChangeNotifierProvider( // ห่อ MyApp ด้วย Provider
      create: (context) => CounterProvider(), // สร้างอินสแตนซ์ของ
Provider
      child: const MyApp(), // MyApp เป็น child
    ),
  );
}
```

- **ผลลัพธ์ที่คาดหวัง (Code):** แอปพลิเคชันของคุณตอนนี้ "provide" อินสแตนซ์ของ CounterProvider ให้กับ widget tree ทั้งหมดที่อยู่ใต้ MyApp. วิดเจ็ตต่างๆ ภายใน MyApp (รวมถึง MyHomePage) จะสามารถเข้าถึง CounterProvider นี้ได้.

ส่วนที่ 4: Refactor UI (Consume State)

1. **คำแนะนำ:** ไปที่คลาส _MyHomePageState ใน main.dart.
2. **คำแนะนำ:** ลบตัวแปร state _counter และเมธอด _incrementCounter ออกจาก _MyHomePageState. เราไม่ต้องการ setState สำหรับตัวนับอีกต่อไปแล้ว.

```
class _MyHomePageState extends State<MyHomePage> {
  // ลบ: int _counter = 0;
  // ลบ: void _incrementCounter() { ... }

  @override
  Widget build(BuildContext context) {
    // ... UI ...
  }
}
```

3. **คำแนะนำ:** ในเมธอด build ของ _MyHomePageState, ค้นหาวิดเจ็ต Text ที่แสดงค่าตัวนับ.
4. **คำแนะนำ:** แทนที่การเข้าถึง _counter โดยตรงด้วยการใช้ context.watch<CounterProvider>(). นี่จะ:
   - ค้นหา CounterProvider ที่ใกล้ที่สุดใน tree.
   - คืนค่าอินสแตนซ์ของ CounterProvider.
   - ทำให้ _MyHomePageState rebuild เมื่อ CounterProvider เรียก notifyListeners().
5. **คำแนะนำ:** เข้าถึง getter count บนอินสแตนซ์ provider ที่ได้มา.

```
@override
Widget build(BuildContext context) {
  // ใช้ context.watch เพื่อรับค่าและ listen การเปลี่ยนแปลง
  final counterValue = context.watch<CounterProvider>().count;

  return Scaffold(
    appBar: AppBar(
      title: Text(widget.title),
    ),
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
```

```
        children: <Widget>[
          const Text(
            'You have pushed the button this many times:',
          ),
          Text(
            '$counterValue', // แสดงค่าจาก Provider
            style: Theme.of(context).textTheme.headlineMedium,
          ),
        ],
      ),
    ),
    floatingActionButton: FloatingActionButton(
      // onPressed จะแก้ไขในส่วนถัดไป
      onPressed: () { print('ยังไม่ได้เชื่อมต่อ Provider'); },
      tooltip: 'Increment',
      child: const Icon(Icons.add),
    ),
  );
}
```

- **ผลลัพธ์ที่คาดหวัง (Code):** UI (Text widget) ตอนนี้ผูกกับ state ใน CounterProvider ผ่าน context.watch.

ส่วนที่ 5: Refactor Action (Trigger State Change)

1. **คำแนะนำ:** ในเมธอด build ของ _MyHomePageState, ค้นหา onPressed callback ของ FloatingActionButton.
2. **คำแนะนำ:** แทนที่ callback เดิมด้วยการใช้ context.read<CounterProvider>(). เราใช้ read ที่นี่เพราะ:
   - เราไม่ต้องการให้ปุ่ม rebuild เมื่อค่าตัวนับเปลี่ยน.
   - เราเพียงต้องการเข้าถึง provider *หนึ่งครั้ง* เพื่อเรียกเมธอดของมัน.
3. **คำแนะนำ:** เรียกเมธอด increment() บนอินสแตนซ์ provider ที่ได้จาก context.read. ใช้ arrow function () => ... เพื่อความกระชับ.

```
floatingActionButton: FloatingActionButton(
  // ใช้ context.read เพื่อเรียกเมธอดโดยไม่ต้อง listen
  onPressed: () => context.read<CounterProvider>().increment(),
```

```
  tooltip: 'Increment',
  child: const Icon(Icons.add),
),
```

- **ผลลัพธ์ที่คาดหวัง (Code):** ปุ่ม FloatingActionButton ตอนนี้เชื่อมต่อกับเมธอด increment ใน CounterProvider ผ่าน context.read.

ส่วนที่ 6: ทดสอบ

1. **คำแนะนำ:** ทำ Hot Restart (Shift+R) หรือรีสตาร์ทแอปพลิเคชันเต็มรูปแบบ.
2. **คำแนะนำ:** สังเกตค่าเริ่มต้นของตัวนับ (ควรเป็น 0).
3. **คำแนะนำ:** แตะ FloatingActionButton หลายๆ ครั้ง.
   - **ผลลัพธ์ที่คาดหวัง (UI & พฤติกรรม):** ค่าตัวนับที่แสดงบนหน้าจอควรเพิ่มขึ้นทุกครั้งที่คุณแตะปุ่ม (+). สิ่งนี้ยืนยันว่า:
     - ChangeNotifierProvider กำลังทำงาน.
     - context.read กำลังเรียกเมธอด increment ใน provider.
     - notifyListeners() ใน provider กำลังถูกเรียก.
     - context.watch ใน UI กำลังรับการแจ้งเตือนและทำให้ Text widget rebuild ด้วยค่าใหม่.
     - ทั้งหมดนี้เกิดขึ้นโดยไม่ต้องใช้ setState ใน _MyHomePageState เลย!

ส่วนที่ 7: (ทางเลือก) แสดงการแชร์ State

1. **คำแนะนำ:** สร้างวิดเจ็ตใหม่ (Stateless) ที่แสดงค่าตัวนับอีกครั้ง.

```
// เพิ่มคลาสนี้ใน main.dart หรือไฟล์แยกต่างหาก
class CounterDisplay extends StatelessWidget {
  const CounterDisplay({super.key});

  @override
  Widget build(BuildContext context) {
    // ใช้ watch อีกครั้งเพื่อแสดงค่าและ listen การเปลี่ยนแปลง
    final count = context.watch<CounterProvider>().count;
    print('CounterDisplay rebuild: $count'); // ดูว่า rebuild เมื่อไหร่
    return Text(
      'ค่าจาก Provider: $count',
      style: Theme.of(context).textTheme.titleMedium,
    );
  }
}
```

2. **คำแนะนำ:** ในเมธอด build ของ _MyHomePageState, เพิ่มอินสแตนซ์ของ CounterDisplay เข้าไปใน Column ใน body (เช่น ใต้ Text ที่แสดงค่าตัวนับหลัก).

```
// ภายใน Column ของ _MyHomePageState -> build
children: <Widget>[
  const Text( /* ... */ ),
  Text( /* ... ตัวนับหลัก ... */ ),
  const SizedBox(height: 20), // เพิ่มช่องว่าง
  const CounterDisplay(), // เพิ่มวิดเจ็ตแสดงผลตัวที่สอง
],
```

3. **คำแนะนำ:** ทำ Hot Restart.
4. **คำแนะนำ:** แตะ FloatingActionButton หลายๆ ครั้ง.
    - ผลลัพธ์ที่คาดหวัง (UI & พฤติกรรม): คุณจะเห็นค่าตัวนับแสดงในสองตำแหน่งบนหน้าจอ. เมื่อคุณแตะปุ่ม ค่า ทั้งสอง จะอัปเดตพร้อมกัน สิ่งนี้แสดงให้เห็นว่า CounterProvider กำลังจัดการ state ส่วนกลางเพียงแห่งเดียว และวิดเจ็ตใดๆ ที่ watch มันอยู่จะได้รับอัปเดตโดยอัตโนมัติ. คุณจะเห็นข้อความ "CounterDisplay rebuild" ใน console ทุกครั้งที่ค่าเปลี่ยน.
      Image Placeholder แสดง UI ตัวนับพร้อมค่าที่แสดงสองตำแหน่ง

ยินดีด้วย! คุณได้ refactor แอปพลิเคชันตัวนับ setState แบบดั้งเดิมให้ใช้ Provider สำหรับการจัดการ state สำเร็จแล้ว คุณได้เรียนรู้วิธีสร้าง ChangeNotifier, provide state ด้วย ChangeNotifierProvider, และ consume/trigger state โดยใช้ context.watch และ context.read. นี่เป็นก้าวสำคัญสู่การเขียนแอป Flutter ที่มีโครงสร้างดีและปรับขนาดได้!